



ÉCOLE NORMALE SUPÉRIEURE DE
L'ENSEIGNEMENT TECHNIQUE MOHAMMEDIA

PROJET STRUCTURES DE DONNÉES

Jeu Tetriste

DÉPARTEMENT MATHÉMATIQUES ET INFORMATIQUE
GÉNIE DU LOGICIEL ET DES SYSTÈMES INFORMATIQUES DISTRIBUÉS
(GLSID)

Réalisé par :

FAIK Youssef
ELMOUTAOUAKIL Abderahim
IDALI OUALI Imad

Encadré par :

M. QBADOU Mohammed

Année Universitaire : 2023-2024

Table des matières

1	Introduction	3
1.1	Contexte du projet	3
1.2	Objectifs	3
2	Fondements théoriques	4
2.1	Concept de tetrisme	4
2.2	Le choix du bibliothèque raylib	4
2.3	Langage C++ : Avantages et utilisation dans le développement de jeux . .	4
3	Conception	5
3.1	Architecture du jeu	5
3.2	Schémas et diagrammes UML	6
3.2.1	Diagramme de cas d'utilisation	6
3.2.2	Diagramme de classe	6
3.2.3	Diagramme des activités	7
4	Implémentation	8
4.1	Environnement de développement	8
4.2	Développement des fonctionnalités principales	9
4.3	Stratégie de test	9
5	Résultats et captures d'écran	10
5.1	Menu principale	10
5.2	Partie du jeu	10
5.3	Les contrôles du jeu pour aider le joueur	11
5.4	Les règles du jeu	11
5.5	Les niveau du jeu	12
5.6	Système d'enregistrement du score	12
6	Conclusion	13

Table des figures

1	Schéma de l'architecture	5
2	Diagramme de cas d'utilisation	6
3	Diagramme de classe	6
4	Diagramme des activités	7
5	Schéma des branches GitHub	8
6	Menu principale	10
7	Partie du jeux	10
8	Les contrôles du jeux pour aider le joueur	11
9	Les règles du jeux	11
10	Les niveau du jeux	12
11	Système d'enregistrement du score	12

1 Introduction

1.1 Contexte du projet

Le projet Tetrisme est une initiative personnelle dans le cadre de l'apprentissage de la programmation en C++. Il vise à mettre en pratique les compétences acquises en travaillant avec les concepts de C++ et en apprenant à travailler avec une bibliothèque graphique. Inspiré du jeu classique Tetris, ce projet offre une opportunité d'explorer le développement de jeux vidéo en utilisant ces technologies. En se concentrant sur la manipulation d'objets, la gestion des événements et la création d'une interface utilisateur interactive, Tetrisme représente un défi enrichissant pour tout développeur aspirant à maîtriser les bases du développement de jeux en C++ avec une approche graphique.

1.2 Objectifs

- Développer le jeu Tetrisme avec des fonctionnalités basiques telles que l'affichage ASCII, la manipulation des pièces et la suppression de triplets.
- Intégrer un système de score pour récompenser les actions du joueur, avec une attention particulière aux décalages et aux suppressions de triplets.
- Limiter la présence de pièces sur le plateau de jeu à 15 pour rendre le jeu plus stratégique et équilibré.
- Mettre en place un mécanisme de sauvegarde/chargement de parties pour une expérience utilisateur fluide.
- Ajouter un système de difficulté permettrait de rendre l'expérience de jeu plus dynamique
- Implémenter un système de meilleurs scores pour inciter à la compétition et à l'amélioration des performances.
- Faciliter le jeu en affichant les prochaines pièces à venir, tout en envisageant une interface graphique simple pour une meilleure expérience utilisateur.

2 Fondements théoriques

2.1 Concept de tetriste

Le processus de tetriste consiste à placer des pièces de différentes formes sur un plateau de jeu, dans le but de former des uplets complets. Lorsqu'une uplet est entièrement remplie par des pièces de la même forme ou couleur, ces pièces disparaissent du plateau, ce qui permet de libérer de l'espace pour de nouvelles pièces.

2.2 Le choix du bibliotheque raylib

Raylib est une bibliothèque de développement de jeux et d'applications graphiques qui offre une approche simple et efficace pour la création de jeux 2D et 3D. En choisissant Raylib, je bénéficie d'une grande variété d'outils et de fonctionnalités pour gérer les graphismes, les entrées utilisateur et les interactions nécessaires à la réalisation d'un jeu interactif. De plus, Raylib est conçu pour être facile à prendre en main, avec une documentation complète et des exemples pratiques, ce qui accélère le processus de développement et permet de se concentrer davantage sur la conception du jeu plutôt que sur des détails techniques. En utilisant Raylib, je suis assuré de bénéficier d'une performance optimale sur différentes plateformes, ce qui garantit une expérience de jeu fluide et agréable pour les joueurs.

2.3 Langage C++ : Avantages et utilisation dans le développement de jeux

Le langage C++ est souvent privilégié dans le développement de jeux en raison de ses nombreux avantages, en particulier sa capacité à mettre en œuvre la programmation orientée objet (POO). Cette approche permet une organisation claire et modulaire du code, facilitant la gestion de la logique du jeu, des interactions entre les différents éléments, et la réutilisation du code. La POO offre également une abstraction efficace, permettant aux développeurs de concevoir des systèmes complexes de manière plus intuitive. De plus, C++ offre un contrôle précis sur la gestion de la mémoire, ce qui est crucial pour optimiser les performances dans les jeux nécessitant des calculs intensifs en temps réel. En combinant ces fonctionnalités avec la flexibilité et la puissance intrinsèques à C++, les développeurs peuvent créer des jeux de qualité tout en maintenant un code bien structuré et facilement évolutif.

3 Conception

3.1 Architecture du jeu

Dans notre architecture de jeu, nous avons adopté une approche basée sur des classes pour garantir une gestion efficace des composants. Voici les principales classes utilisées dans notre projet :

- **ColorInfo** : Cette classe représente les informations relatives à une couleur spécifique dans le jeu. Elle stocke le nombre d'éléments de cette couleur ainsi qu'un pointeur vers le premier élément associé à cette couleur dans la liste chaînée des pièces.
- **FormInfo** : Similaire à ColorInfo, cette classe stocke les informations sur une forme spécifique dans le jeu, telles que le nombre d'éléments et un pointeur vers le premier élément de cette forme dans la liste des pièces.
- **Node** : Cette classe représente un nœud dans la liste chaînée des pièces. Chaque nœud contient une pièce de jeu ainsi que des pointeurs vers les nœuds suivants et précédents pour les couleurs et les formes correspondantes.
- **Piece** : Une classe qui définit une pièce de jeu, caractérisée par sa couleur et sa forme.
- **Plateau** : Cette classe représente le plateau de jeu où les pièces sont placées et manipulées. Elle gère divers aspects du jeu, tels que la taille du plateau, le score du joueur, les pièces en attente, etc.

En plus de ces classes, nous utilisons des énumérations pour définir les couleurs et les formes disponibles dans le jeu, ainsi que des constantes pour déterminer les côtés de décalage et les types de décalage possibles. Notre architecture modulaire permet une gestion efficace de la complexité du jeu en séparant les différentes fonctionnalités et en les organisant de manière logique et cohérente. Chaque classe est responsable d'un aspect spécifique du jeu, ce qui facilite la compréhension, la maintenance et l'évolutivité du code.

Voici un simple schéma qui montre l'architecture :

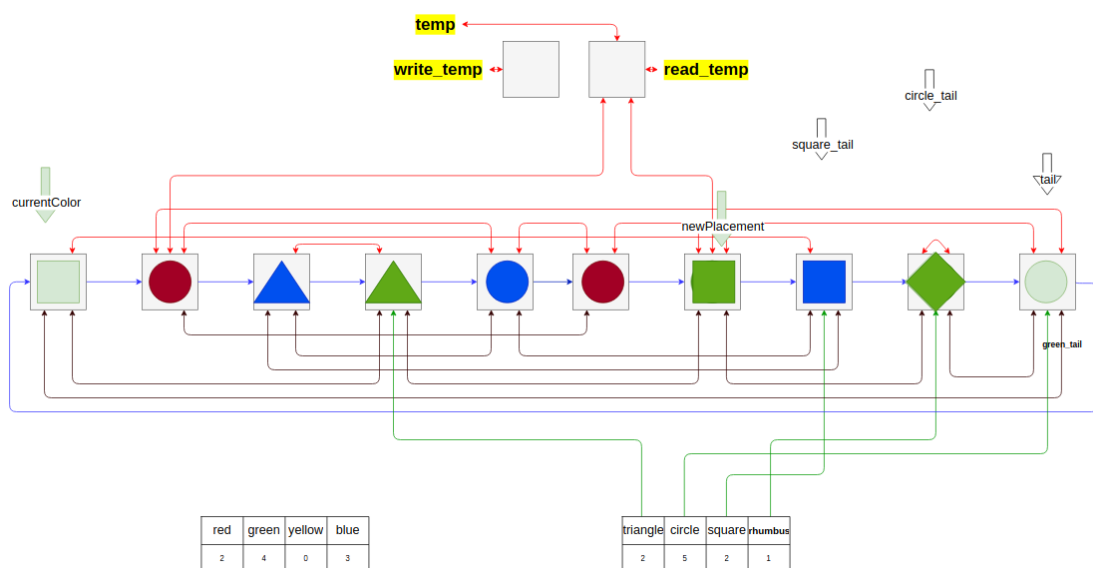


FIGURE 1 – Schéma de l'architecture

3.2 Schémas et diagrammes UML

3.2.1 Diagramme de cas d'utilisation

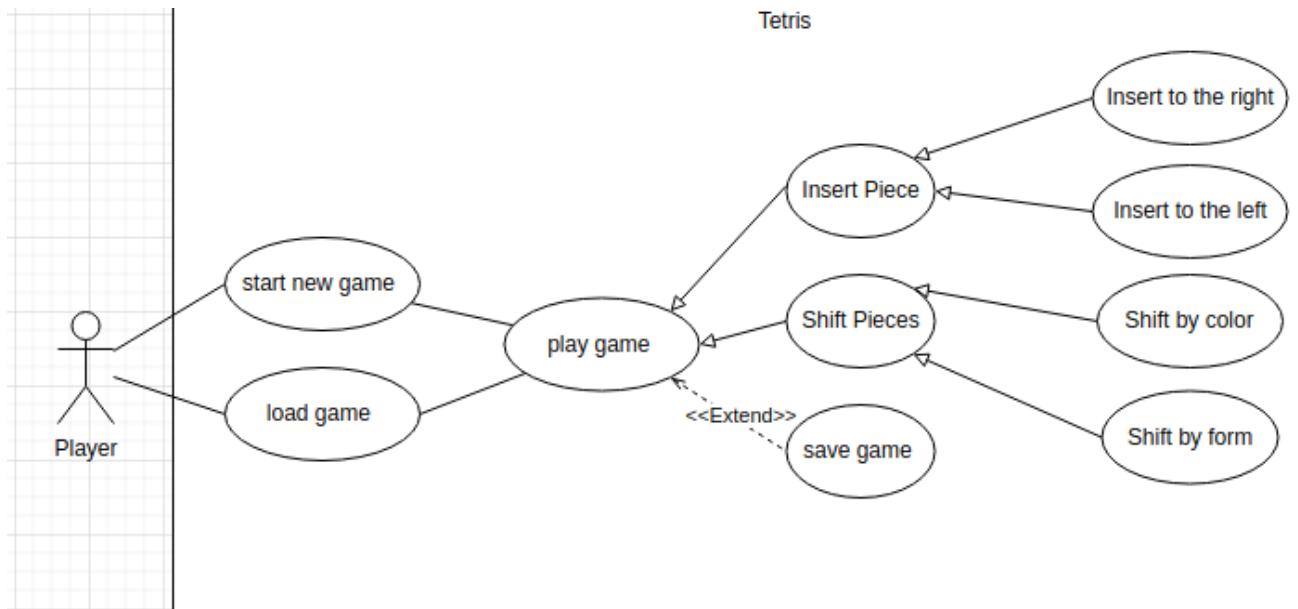


FIGURE 2 – Diagramme de cas d'utilisation

3.2.2 Diagramme de classe

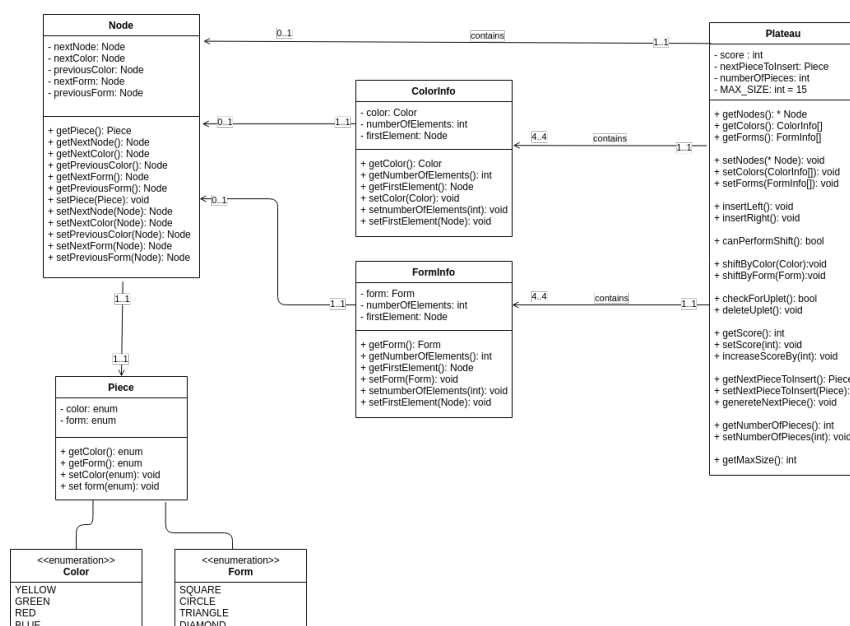


FIGURE 3 – Diagramme de classe

3.2.3 Diagramme des activités

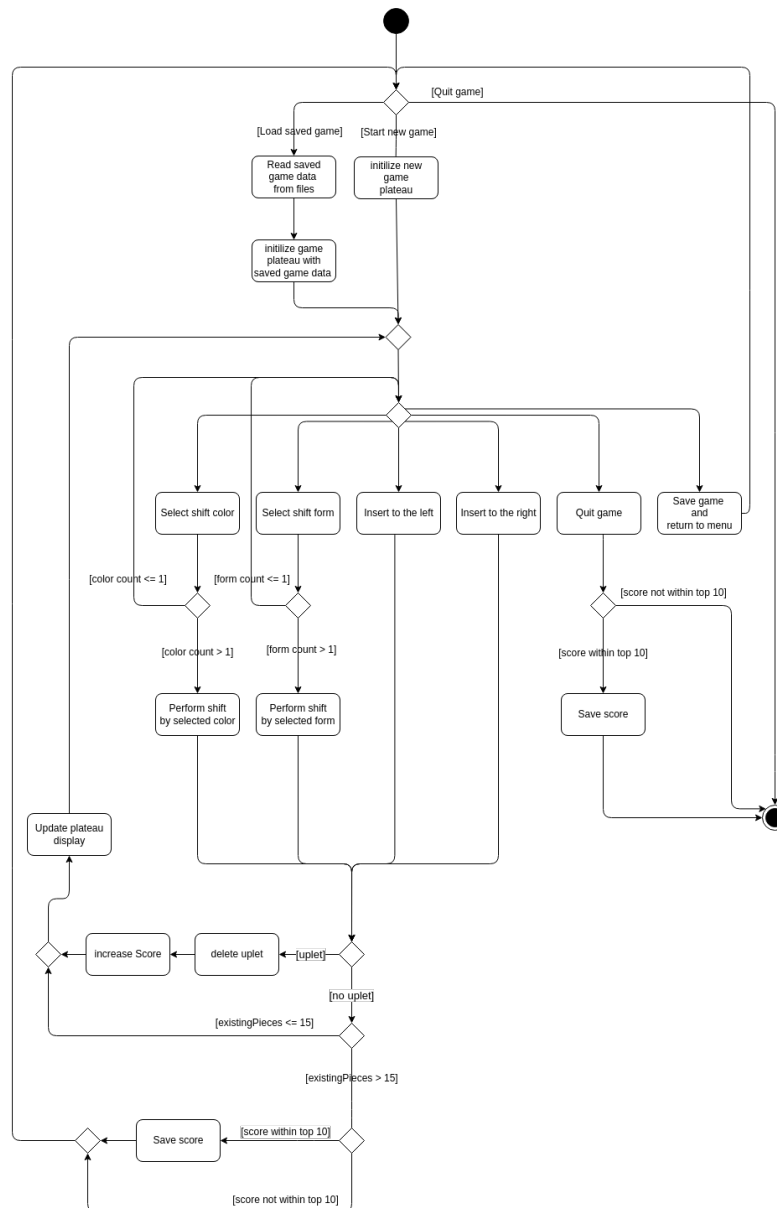


FIGURE 4 – Diagramme des activités

4 Implémentation

4.1 Environnement de développement

Pour le développement de notre jeu Tetriste, nous avons choisi Visual Studio Code comme principal environnement de développement, en raison de sa compatibilité avec le C++ et de sa légèreté. Ses fonctionnalités dédiées au langage, telles que la coloration syntaxique avancée et les outils de débogage, ont facilité notre travail.

Pour la compilation, nous avons utilisé le compilateur g++ pour sa compatibilité avec les normes modernes du C++ et sa capacité d'optimisation. Sous Linux Ubuntu, nous avons bénéficié d'outils intégrés pour la gestion de la mémoire et la détection d'erreurs. Cette combinaison d'outils et d'environnements nous a permis de développer efficacement notre jeu tout en garantissant sa qualité et sa stabilité.

Pour la gestion de notre code source et la collaboration entre membres de l'équipe, nous avons choisi Git comme système de contrôle de version, avec GitHub comme plateforme d'hébergement. Git nous permet de suivre les modifications apportées au code, de gérer les branches de développement et de fusionner les contributions de manière efficace.

Schéma des branches GitHub :

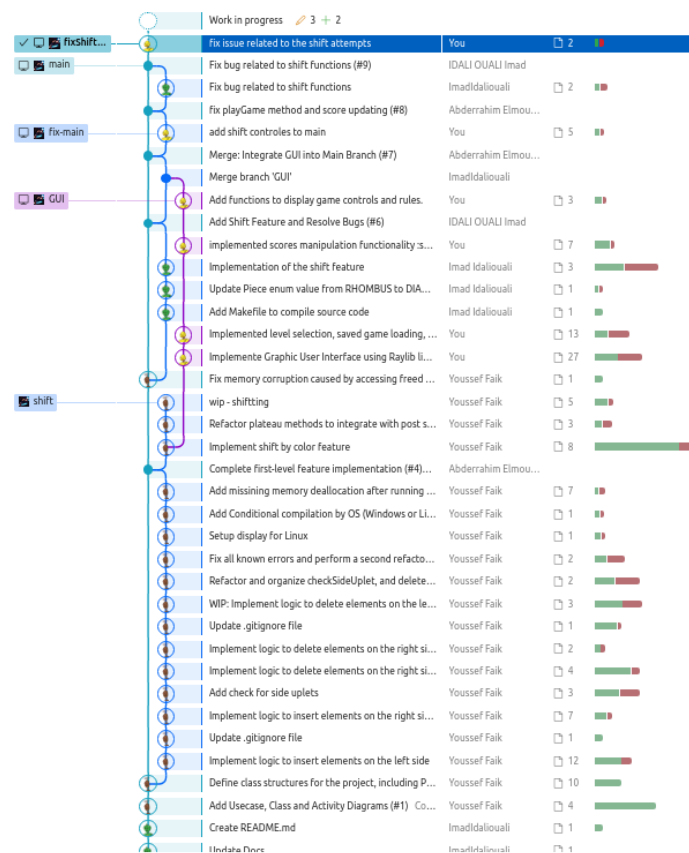


FIGURE 5 – Schéma des branches GitHub

4.2 Développement des fonctionnalités principales

- **Génération aléatoire de pièces :** Nous avons travaillé sur des fonctions permettant de générer aléatoirement des pièces avec différentes formes et couleurs. En utilisant les fonctionnalités de génération de nombres aléatoires de la bibliothèque standard C++, nous avons assuré une diversité et une variabilité dans le jeu.
- **Insertion et suppression de pièces :** mis en place des méthodes pour insérer et supprimer des pièces sur le plateau de jeu. Notre objectif était de maintenir la cohérence des connexions entre les pièces de même couleur ou forme, assurant ainsi une expérience de jeu fluide et logique.
- **Décalage :** créer un système de décalage basé sur les couleurs, permettant aux pièces de se déplacer latéralement en fonction de leur couleur respective. Cette fonctionnalité stratégique ajoute une nouvelle couche de complexité au jeu, incitant les joueurs à planifier leurs mouvements en tenant compte des couleurs des pièces sur le plateau.
- **Enregistrement de l'état du jeu :** Nous avons créé des fonctions pour sauvegarder l'état actuel du jeu dans un fichier texte. Cela inclut les pièces actuellement présentes sur le plateau, leur position et leurs propriétés telles que la couleur et la forme. À chaque étape importante du jeu, telles que l'insertion ou la suppression de pièces.
- **Enregistrement des scores des joueurs :** Lorsqu'un joueur termine une partie, nous lui demandons de saisir son nom. Nous enregistrons ensuite ce nom ainsi que son score dans un autre fichier texte dédié aux scores. Chaque joueur a une entrée dans ce fichier, contenant son nom et son score.
- **Affichage graphique :** exploité les fonctionnalités de rendu de SDL2 pour afficher les pièces du jeu, ainsi que le fond d'écran et d'autres éléments visuels. Cette collaboration nous a permis de créer une interface visuellement attrayante pour les joueurs.
- **Gestion des événements utilisateur :** pour mettre en place un système robuste de gestion des événements utilisateur. Cela nous a permis de détecter les pressions de touches du clavier et autres interactions, offrant ainsi aux joueurs une expérience de jeu intuitive et fluide.

4.3 Stratégie de test

Pour tester les fonctionnalités du jeu, nous avons initialement développé une petite interface console. Cette interface nous a permis de visualiser l'état des pointeurs à chaque étape du jeu, que ce soit lors du décalage, de l'insertion ou de la suppression des pièces. Cela nous a offert une vue détaillée et précise de l'évolution du jeu, facilitant ainsi le processus de débogage et d'optimisation des fonctionnalités.

Concernant la mémoire nous avons utilisé un outil puissant sous Linux appelé Valgrind. Cet outil nous a permis de détecter et de corriger les fuites de mémoire potentielles, assurant ainsi une gestion efficace de la mémoire pendant l'exécution du jeu. Grâce à Valgrind, nous avons pu effectuer des tests approfondis pour garantir que notre jeu fonctionne de manière fiable et sans fuites de mémoire.

5 Résultats et captures d'écran

Quelques captures d'écran du jeu Tetrisme en action :

5.1 Menu principale

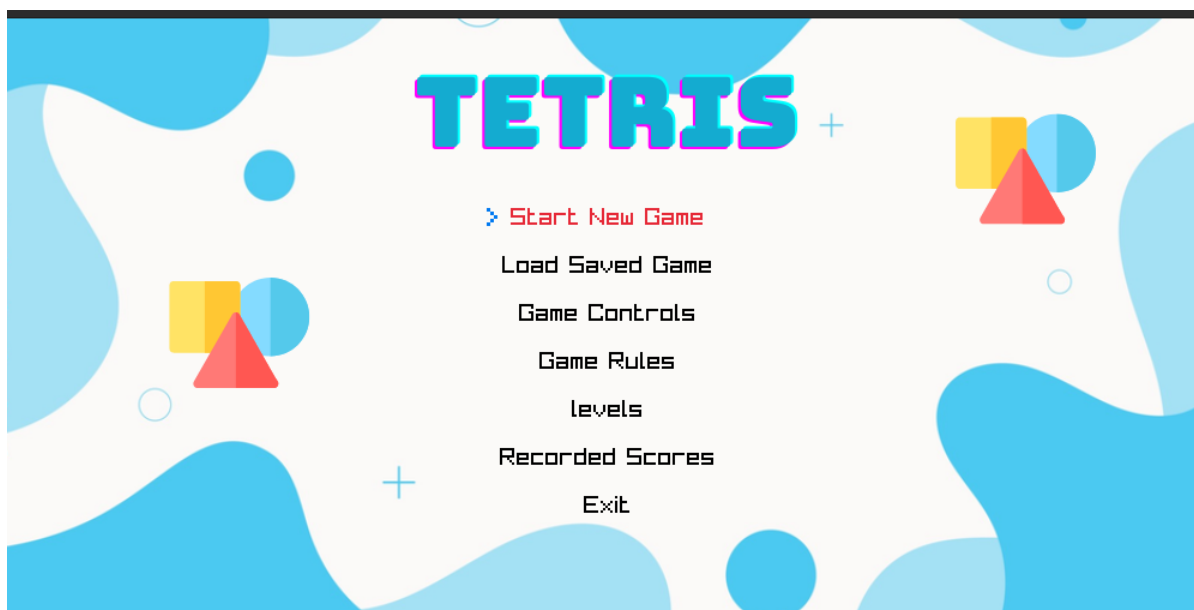


FIGURE 6 – Menu principale

5.2 Partie du jeu

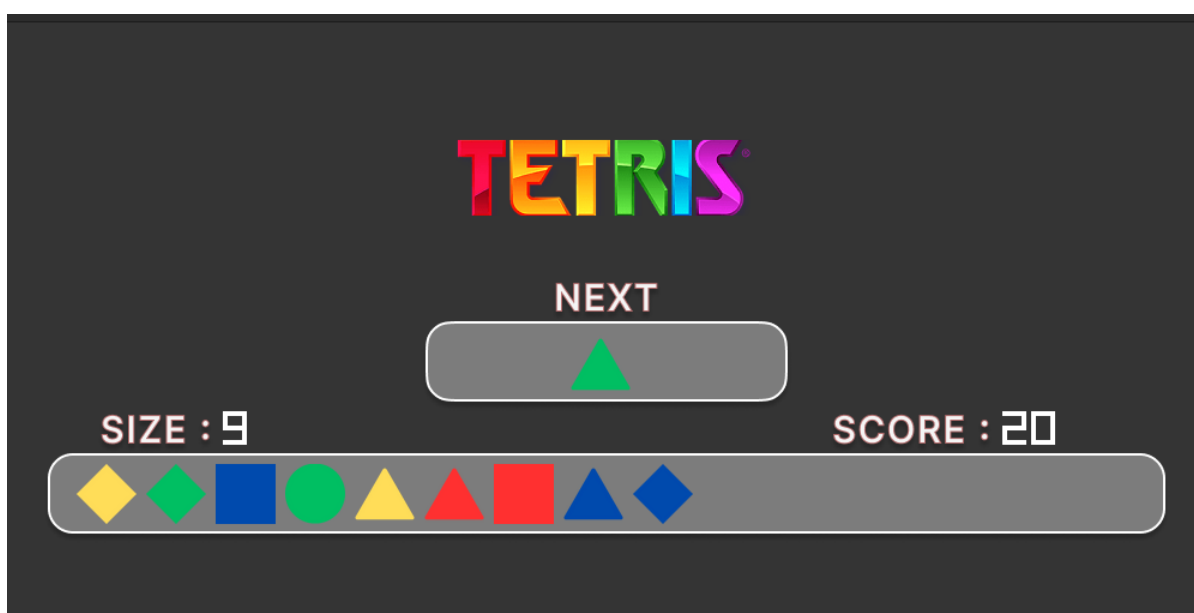


FIGURE 7 – Partie du jeu

5.3 Les contrôles du jeux pour aider le joueur

Controls

- Use Left and Right arrow keys to insert the next piece
- Press B to shift by blue color
- Press G to shift by green color
- Press R to shift by red color
- Press Y to shift by yellow color
- Press C to shift by circle form
- Press S to shift by square form
- Press T to shift by triangle form
- Press D to shift by diamond form

FIGURE 8 – Les contrôles du jeux pour aider le joueur

5.4 Les règles du jeux

Rules

The goal of the game is to create uplets of shapes or colors (square, triangle, diamond, circle)(red, green, yellow, blue).

Each uplet you create, earns you points. The number of points you earn depends on the difficulty level you choose.

Easy:

Uplet size: 2

Maximum pieces on the plateau: 6

Points awarded per uplet: 4

Medium:

Uplet size: 3

Maximum pieces on the plateau: 10

Points awarded per uplet: 8

Hard:

FIGURE 9 – Les règles du jeux

5.5 Les niveau du jeux

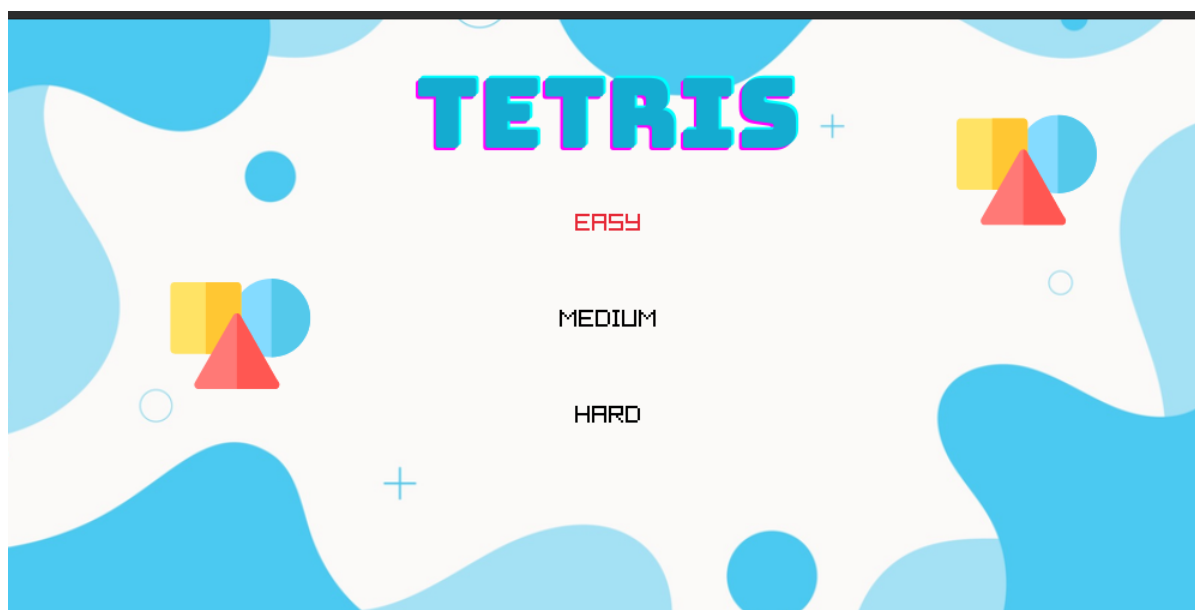


FIGURE 10 – Les niveau du jeux

5.6 Système d'enregistrement du score

SCORE	PLAYER
320	- ERENYEAGER
230	- enregistrer
195	- hug_From_Bug
180	- Amine
140	- Youssef
130	- Abderrahim
55	- imad
45	- andiano
40	- squareShift
40	- syzyzyz

FIGURE 11 – Système d'enregistrement du score

6 Conclusion

Ce projet collaboratif a été une opportunité pour nous de renforcer nos compétences en programmation en travaillant avec des structures de données avancées et des concepts de l'OOP. Nous avons développé une compréhension approfondie de la manière dont les classes, les objets et les méthodes peuvent être utilisés pour modéliser efficacement et de manière organisée des systèmes complexes. En travaillant ensemble, nous avons amélioré notre capacité à faire des conceptions en utilisant UML, à utiliser des outils tels que Git et GitHub, et à concevoir des solutions élégantes et efficaces, puis à les mettre en œuvre en utilisant le langage C++.

Références

- [1] Code source : github.com/AbderrahimeEl/Tetris